

CSE 4102: Computer Graphics and Image Processing Laboratory

3D TARGET SHOOTER GAME

By

Kazi Rifat Al Muin

Roll: 2107042



Submitted To

Md Tajmilur Rahman

Lecturer

&

Md. Mubtashim Abrar Nihal

Lecturer

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Khulna 9203, Bangladesh

October, 2026

1 Introduction and Objectives

Abstract. 3D Target Shooter is a C++17/OpenGL 3.3 game built from one shared cube. Seven levels combine target motion, aiming, shooting and NPC penalties. Three shading modes and day/night lighting expose the underlying graphics equations. The same model transforms determine object appearance, aiming and collision [1].

1.1 Relationship to the proposed project

The proposal specifies an arena, aimed gun, circular targets, stands, bullets, scenery, hit effects, lighting and camera controls. These functions are present. Curved geometry is simplified: barrels and bullets are cuboids; targets use 32 thin cubes with fragment-shader rings. The table distinguishes functionality from the proposed primitive shapes.

1.2 Objectives and proposal fulfilment

The objectives are to construct a complete arena from a reusable primitive, demonstrate affine transformations and illumination, and connect those calculations to interactive aiming, motion and collision.

Proposed feature	Final status	Implementation	Evidence
Arena and scenery	Fully implemented	Floor, walls, crates, platforms	Arena/Cargo builders; Sec. 3.2–3.4
Gun, bullets, circular targets	Partially implemented	Cuboids and 32 target slices replace curved meshes	Weapon/Target builders; Sec. 3.5–4.1
Motion and hit effects	Fully implemented	Sinusoids, rotation, recoil, fragments	Sec. 2.2 and 4
Lighting and shading	Fully implemented	Ambient, sun, points, spots; three shading modes	Shader comparisons; Sec. 7
Views and controls	Fully implemented	Player/arena/side/free views and mouse aiming	Camera and input logic; Sec. 5

World coordinates use meters; angles use degrees unless stated otherwise.

2 Gameplay and Level Design

2.1 Game modes, scoring and interaction



Player view: aiming and HUD.



Arena view: walls, lanes and targets.

Figure 1: The finished game: one scene viewed from the player and arena cameras.

The player selects a mode, aims at a target front and fires. Challenge has a 1.5-second level introduction; clearing level 7 shows victory. Menus, pause and results suspend simulation. Nearby cover can block a muzzle even when the camera sees a target.

Mode	Rules and persistence
Challenge	Seven sequential levels; accumulated score/time; eligible results after completed levels.
Free	Level-7 environment; 180-second session; targets return after 30 seconds; save at expiry.
Developer	Directly inspect any level; restart and diagnostics; no competitive record.
Bird's-Eye	Level-7 overview and safe click-to-observe camera; noncompetitive, no shooting.
Practice	Seven sandbox targets; free movement; 1.8-second target respawn; separate scoring.

Hit rules. A target has 60 health. Ring index $r = 0, \dots, 5$ gives integer damage $D(r) = 60/(r+1)$, hence 60, 30, 20, 15, 12, 10: respectively 1–6 equal-ring shots to destroy a fresh target. Back and edge contacts block a shot but do not award ring damage. All pellets from one trigger share a shot ID; the target records the *maximum* ring damage from that shot rather than summing pellets. Level modes award +150 for a one-shot target or +100 otherwise, with –100 per bird and –200 per human fatal hit. Practice adds applied damage plus a 100-point destruction bonus. Negative scores remain valid.

Leaderboards separate modes and normalized names; eligible completion boundaries save best records. CSV parsing supports quoted names and negative scores.

2.2 Seven levels and bounded target motion

Level configuration supplies base position $\mathbf{b}$, amplitude $\mathbf{a}$, signed frequency ω , phase ϕ and spin s . All levels use the same world and gameplay builders. Levels 5–7 extend the preceding configuration rather than implementing separate engines.

Level	Targets	Walking	New difficulty in the actual configuration
1	3	Locked	Static targets at $(-5, 2.4, -18)$, $(0, 2.7, -22)$, $(5, 2.4, -18)$.
2	3	Locked	Center target moves horizontally: base $(0, 3, -31)$, amplitude $(2.3, 0, 0)$.
3	4	Locked	Separate X, Y and Z motion; phase-offset trajectories.
4	6	Enabled	Cargo screens, longer distances, rotating targets and navigable firing positions.
5	8	Enabled	Compound trajectories; eight configured birds.
6	10	Enabled	Faster three-axis trajectories; sixteen configured birds.
7	12	Enabled	Final pair has faster motion/spin; four birds and three humans per target (48/36).

$$p_j(t) = b_j + a_j \sin(t\omega_j + \phi_j), \quad \theta_y(t) = \text{fmod}(ts, 360). \quad (1)$$

For configuration variation v , let $d = -1$ if v is odd, otherwise $d = 1$. The builder sets

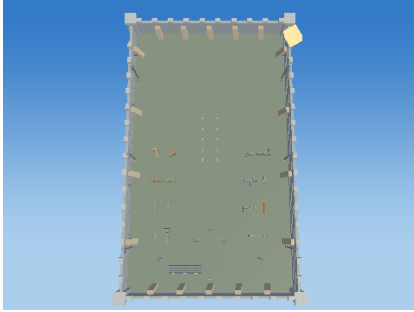
$$\omega = (kd, 1.13k, 0.87kd), \quad \phi = (0.63v, 0.47v, 0.81v), \quad s = s_0 d.$$

Frequencies are radians per second; spin is degrees per second. A zero amplitude fixes that coordinate. For Level 2's center target, $v = 0$, $k = 1.1$, so $x(1) = 2.3 \sin(1.1) \approx 2.0498$, $y = 3$, $z = -31$; motion is bounded by $x \in [-2.3, 2.3]$. Its time-zero center remains $(0, 3, -31)$. Nonzero phases in later levels mean initial positions need not equal the configured base.

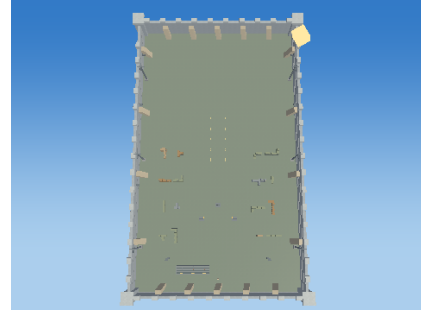
World validation. Locked-player levels check 32 motion samples at .43 s intervals. Walkable levels flood-fill a 51×91 grid and require reachable firing positions. Layout generation retries up to four seeds with increment 7919. Sampling improves playability but does not prove visibility at every instant. Free mode respawns targets after 30 s; Practice after 1.8 s. Configuration is in `src/levels/`; motion is evaluated by `updateTarget`.

2.3 Seven levels from above

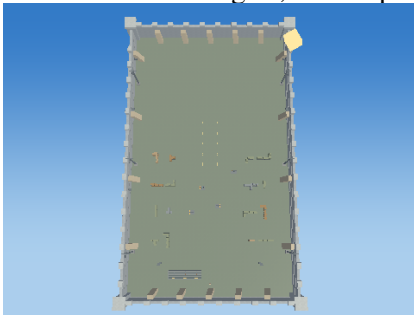
The top-angle views use eye $(0, 140, -25)$, look-at point $(0, 0, -50)$ and a 45° perspective. Every level is shown at its initial simulation state with day lighting and Phong shading. The common viewpoint exposes target distribution, cargo cover, open firing lanes and the added NPC population.



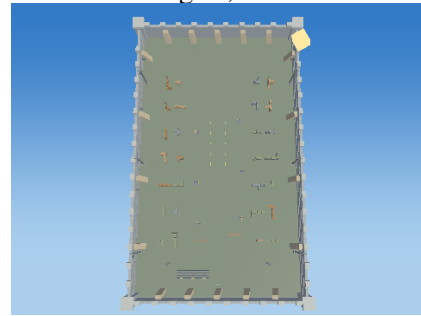
Level 1: three static targets; locked player.



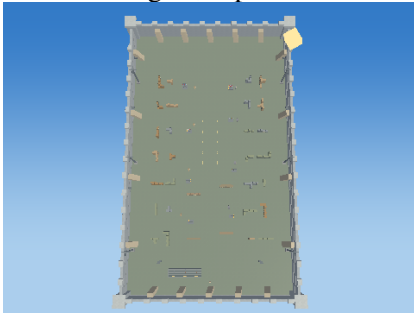
Level 2: three targets; horizontal motion.



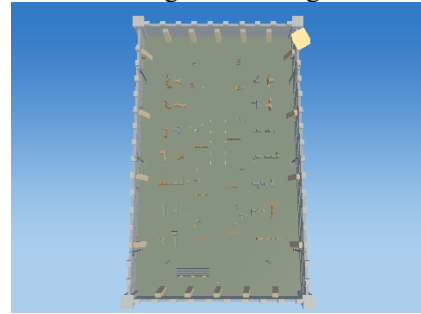
Level 3: four targets; separate motion axes.



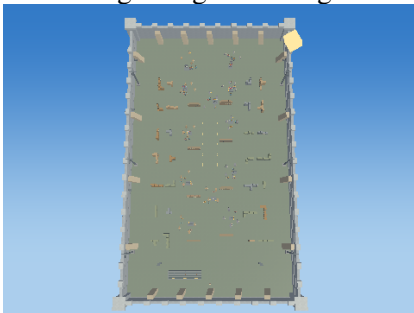
Level 4: six targets; walking and cover.



Level 5: eight targets and eight birds.



Level 6: ten targets and sixteen birds.



Level 7: twelve targets, 48 birds, 36 humans.



Level 7 from the player: depth and occlusion.

Figure 2: Seven level layouts from a common top angle, followed by the final arena at player height.

3 Object Construction and 3D Transformations

3.1 Shared geometry and model transformation

Every world part starts from eight corners $(\pm 0.5, \pm 0.5, \pm 0.5)$. Six faces are expanded to 12 triangles (36 vertices) with separate face normals. One VAO/VBO stores this mesh; every `SceneObject` supplies its transform, color, material and lighting parameters. Assemblies are overlapping draw calls, not Boolean mesh unions. The parent field identifies an assembly; its transform is not automatically multiplied in the renderer.

$$\mathbf{p}_w = M\mathbf{p}_l, \quad M = TR_zR_yR_xHS, \quad \mathbf{p}_c = PVM\mathbf{p}_l, \quad \mathbf{p}_{ndc} = \mathbf{p}_c \cdot xyz / p_c \cdot w. \quad (2)$$

Column vectors are used, and column-major matrix storage is uploaded using `GL_FALSE`. Thus scale is applied first and translation last. With $c = \cos \theta$, $s = \sin \theta$ and $\theta = \pi \theta_{deg} / 180$:

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & h_{xy} & h_{xz} & 0 \\ h_{yx} & 1 & h_{yz} & 0 \\ h_{zx} & h_{zy} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

$$R_x^{3 \times 3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{bmatrix}, \quad R_y^{3 \times 3} = \begin{bmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{bmatrix}, \quad R_z^{3 \times 3} = \begin{bmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Each rotation is extended with the homogeneous fourth row/column. A point has $w = 1$; a direction/offset has $w = 0$ so translation does not affect it. Each stage computes $p'_r = \sum_{c=0}^3 A_{rc} p_c$. All eight cube corners follow this same sequence. **Exact composition loop from `Transform.h`:**

```
Mat4 model = Mat4::identity();
for (const auto& stage : modelTransformStages(t))
    model = stage.matrix * model;
return model;
```

Normal transformation. Scale/shear require $\mathbf{N} = \text{normalize}((A^{-1})^T \mathbf{n}_l)$, $A = M_{3 \times 3}$. For basis columns $\mathbf{a}, \mathbf{b}, \mathbf{c}$, determinant $d = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$ gives normal-matrix columns $(\mathbf{b} \times \mathbf{c}, \mathbf{c} \times \mathbf{a}, \mathbf{a} \times \mathbf{b}) / d$. This preserves perpendicularity under shear. Rasterization then interpolates shader attributes [2].

3.2 Arena, walls and environmental construction

The rectangular floor is 60×100 m. Its center is $(0, -0.1, -50)$ and scale $(60, 0.2, 100)$, so its top lies at $y = 0$. The playable direction is $-Z$. Four continuous 8 m walls enclose the arena; supports and crenellations decorate rather than open the boundary.

Part	Center (x,y,z)	Scale (x,y,z)	Yaw
Floor	$(0, -.1, -50)$	$(60, .2, 100)$	0°
North / South wall	$(0, 4, -100)/(0, 4, 0)$	$(60, 8, 1)$	0°
West / East wall	$(-30, 4, -50)/(30, 4, -50)$	$(100, 8, 1)$	90°
Corner tower	$(\pm 30, 4.75, -100 \text{ or } 0)$	$(3.5, 9.5, 3.5)$	0°
Corner cap	$(\pm 30, 10, -100 \text{ or } 0)$	$(4.2, 1, 4.2)$	0°
End top blocks	$(x, 8.75, -100 \text{ or } 0)$	$(2, 1.5, 1.5)$	0°
Side top blocks	$(\pm 30, 8.75, z)$	$(2, 1.5, 1.5)$	90°

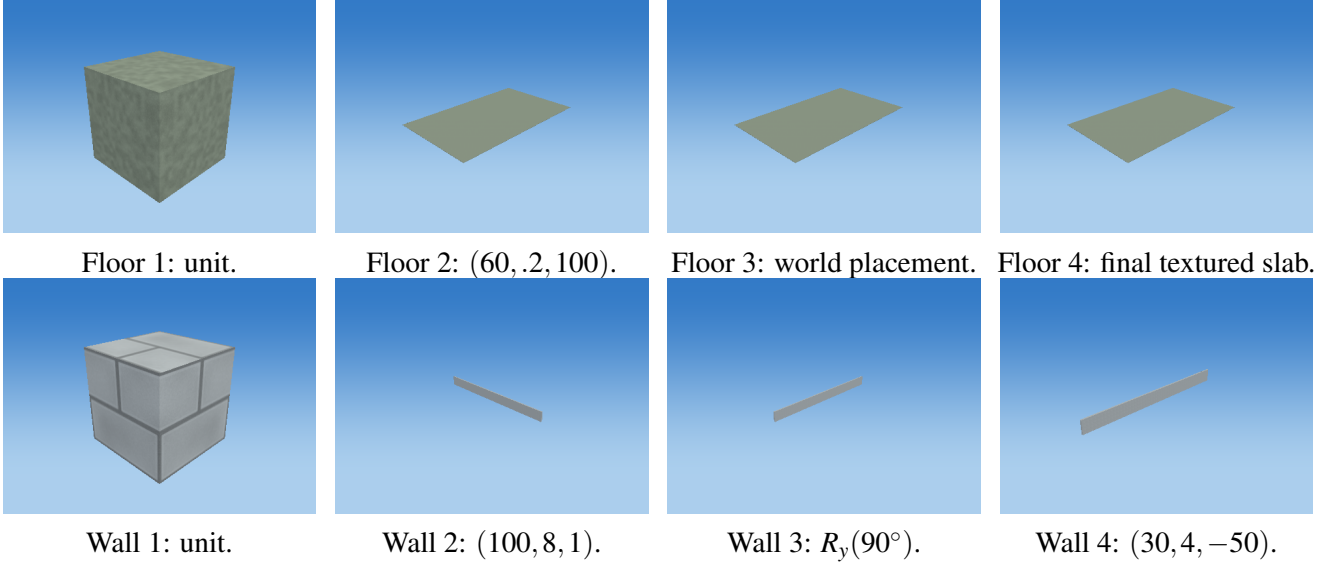


Figure 3: Arena parts are constructed from the shared cube. Scaling sets dimensions, rotation sets orientation, and translation places each component in the arena.

For the east wall, $(.5, .5, .5) \xrightarrow{S} (50, 4, .5) \xrightarrow{R_y(90^\circ)} (.5, 4, -50) \xrightarrow{T} (30.5, 8, -100)$. Its Z span is $[-100, 0]$. Thin route markers sit at $(\pm 2.8, .012, z)$ with scale $(.075, .018, 1.35)$. The equipment platform at $(-9, .55, -8)$ has scale $(12, 1.1, 3)$. Sources: Arena.cpp, Environment.cpp.

3.3 Worked shear, rotation and ground anchoring

The eastern support SUPPORT_E_18 demonstrates a nonidentity shear and Z rotation. Its scale is $(1.5, 6, 2)$, $h_{xy} = 0.22$, rotation $(0, 0, -8^\circ)$ and translation $(28, 2.98332977, -18)$. The builder calculates the Y translation from the transformed corners, so the lowest point touches the ground.

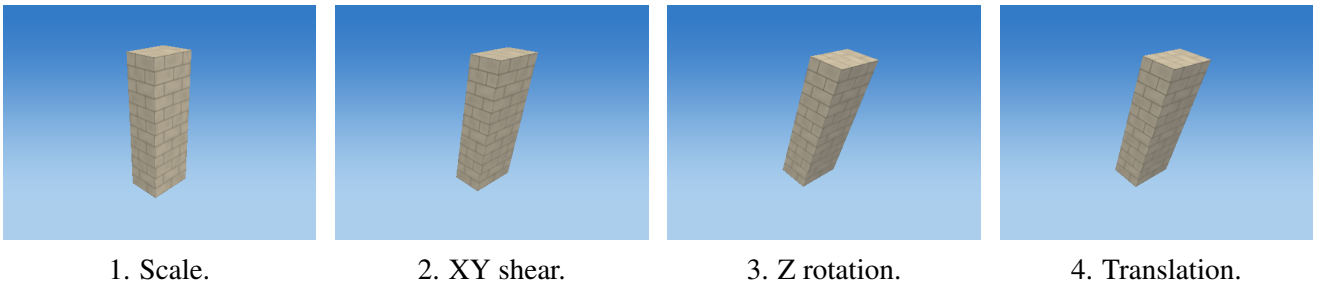


Figure 4: Support: nonidentity stages in order; X/Y rotations are identities.

Operation	Calculation for local corner (.5, .5, .5)	Result
Scale	$(1.5(.5), 6(.5), 2(.5))$	$(.75, 3, 1)$
Shear	$x \leftarrow .75 + .22(3)$	$(1.41, 3, 1)$
X and Y rotations	Both angles are zero	$(1.41, 3, 1)$
Z rotation	$x = c(1.41) - s(3), y = s(1.41) + c(3)$	$(1.8137973, 2.7745701, 1)$
Translation	Add $(28, 2.9833298, -18)$	$(29.8137973, 5.7578999, -17)$

Here $c = \cos(-8^\circ) = 0.9902681$ and $s = -0.1391731$. The resulting matrix, rounded to seven decimals, is

$$M = \begin{bmatrix} 1.4854021 & 2.1421925 & 0 & 28 \\ -.2087597 & 5.7578998 & 0 & 2.9833298 \\ 0 & 0 & 2 & -18 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The vertical extent is $e_y = \frac{1}{2}(|M_{10}| + |M_{11}| + |M_{12}|) \approx 2.9833298$, so setting $t_y = e_y$ makes $y_{min} = t_y - e_y = 0$. The actual code evaluates all eight corners before translating; both calculations describe the same affine box. Collision uses the same composed matrix, including shear.

3.4 Cargo crates: deterministic multipart assembly

The reference human height is 2.1 m; crate side length is $s = 2.1/3 = 0.7$ m. One crate comprises a wood body, two metal bands and a paper inventory plate. The representative is the first Practice crate from seed 2107042. Its yaw is zero and body center is $\mathbf{b} = (-21.23, .35, -20)$. These are actual generated values, not a newly invented layout.

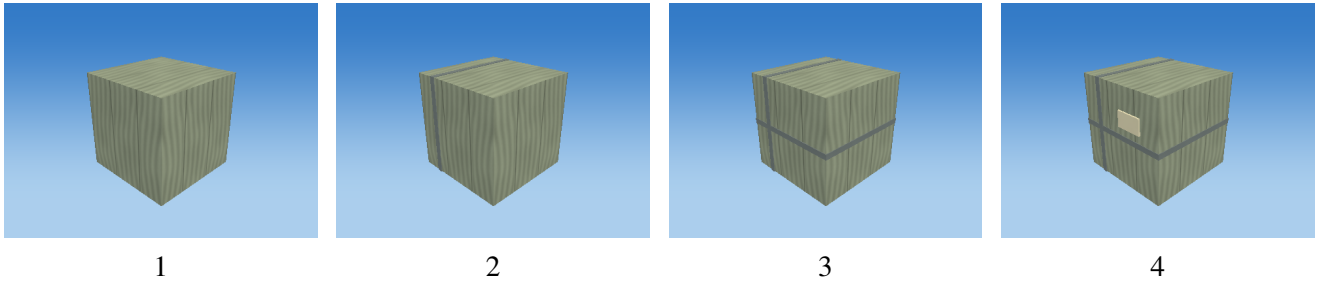


Figure 5: Cargo assembly in builder order: (1) body, (2) vertical band, (3) horizontal band, (4) inventory plate.

Step	Component	World center (m)	Scale (m)	Rotation (deg)
1	Crate	$(-21.23, 0.35, -20)$	$(0.7, 0.7, 0.7)$	$(0, 0, 0)$
2	Crate band	$(-21.426, 0.35, -20)$	$(0.0385, 0.708, 0.708)$	$(0, 0, 0)$
3	Crate band	$(-21.23, 0.35, -20)$	$(0.708, 0.0385, 0.708)$	$(0, 0, 0)$
4	Crate inventory plate	$(-21.125, 0.483, -19.644)$	$(0.189, 0.112, 0.012)$	$(0, 0, 0)$

For every detail, $\mathbf{p} = \mathbf{b} + R_y(\theta)\mathbf{o}$ and its yaw is θ . The exact offset/size rules are

Part	Local offset $\mathbf{o}$	Size
Vertical band	$(-.28s, 0, 0) = (-.196, 0, 0)$	$(.055s, s + .008, s + .008)$
Horizontal band	$(0, 0, 0)$	$(s + .008, .055s, s + .008)$
Plate	$(.15s, .19s, .5s + .006)$	$(.27s, .16s, .012)$

Thus the plate center is $(-21.125, .483, -19.644)$ and its size is $(.189, .112, .012)$. The 0.008 and 0.006 offsets separate exposed surfaces and avoid placing the visible detail exactly coplanar with the crate face. The body bottom is $.35 - .7/2 = 0$. **Repeatable layout.** `std::mt19937(seed)` generates lanes at $x = -20, -15, 15, 20$ with jitter $(r \bmod 7 - 3) \cdot .08$. Crate spacing is $.735$, rows use $z = -20 - 10 \text{ row}$, runs have $3 + r \bmod 5$ columns and yaw is $90(r \bmod 4)$. Height is $1 + r_1 \bmod 3 + r_2 \bmod 3$ crates. Reserved corridors and target clearance constrain the layout. Source: `Cargo.cpp`.

3.5 Pistol assembly, aiming and recoil

The pistol has 11 permanent cuboids. Its body offset is $(.32, -.25, -.68)$ and scale is $(.22, .22, .56)$; its displayed center is $(-12.68, 1.40, -8.68)$. The grip scale is $(.18, .37, .20)$. This fixed equipment-display example uses camera/anchor $\mathbf{c} = (-13, 1.65, -8)$, yaw -90° , pitch 0° and recoil 0; hence its orientation is identity. Each panel adds exactly one component.

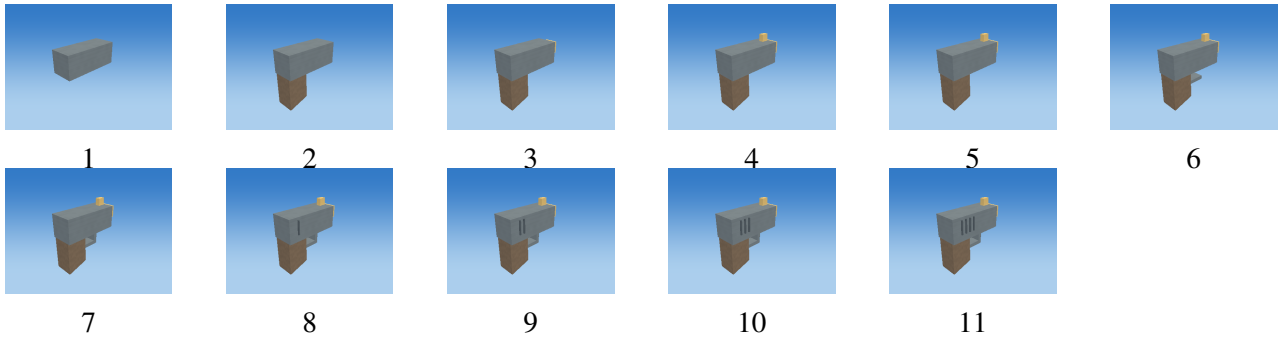


Figure 6: Pistol assembly: body, grip, barrel, sight, muzzle inset, lower trigger guard, front guard, then four side details. Every added cube is retained in the following state.

Step	Component	World center (m)	Scale (m)	Rotation (deg)
1	BODY	$(-12.68, 1.4, -8.68)$	$(0.22, 0.22, 0.56)$	$(0, 0, 0)$
2	GRIP	$(-12.68, 1.19, -8.5)$	$(0.18, 0.37, 0.2)$	$(0, 0, 0)$
3	BARREL	$(-12.68, 1.43, -8.96)$	$(0.13, 0.12, 0.16)$	$(0, 0, 0)$
4	SIGHT	$(-12.68, 1.55, -8.89)$	$(0.045, 0.07, 0.07)$	$(0, 0, 0)$
5	MUZZLE_INSET	$(-12.68, 1.43, -9.043)$	$(0.085, 0.075, 0.012)$	$(0, 0, 0)$
6	TRIGGER_GUARD	$(-12.68, 1.17, -8.68)$	$(0.13, 0.035, 0.19)$	$(0, 0, 0)$
7	GUARD_FRONT	$(-12.68, 1.25, -8.76)$	$(0.13, 0.16, 0.035)$	$(0, 0, 0)$
8	DETAIL_0	$(-12.567, 1.42, -8.49)$	$(0.015, 0.13, 0.013)$	$(0, 0, 0)$
9	DETAIL_1	$(-12.567, 1.42, -8.535)$	$(0.015, 0.13, 0.013)$	$(0, 0, 0)$
10	DETAIL_2	$(-12.567, 1.42, -8.58)$	$(0.015, 0.13, 0.013)$	$(0, 0, 0)$
11	DETAIL_3	$(-12.567, 1.42, -8.625)$	$(0.015, 0.13, 0.013)$	$(0, 0, 0)$

Grip shear. Before rotation, $h_{yz} = -0.3$, so a scaled point becomes $(x, y - .3z, z)$. The grip's local corner $(.5, .5, .5)$ scales to $(.09, .185, .10)$, shears to $(.09, .155, .10)$, and translates by $(-12.68, 1.19, -8.5)$ to $(-12.59, 1.345, -8.4)$. Its matrix is

$$M_{grip} = \begin{bmatrix} .18 & 0 & 0 & -12.68 \\ 0 & .37 & -.06 & 1.19 \\ 0 & 0 & .20 & -8.50 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Aiming and recoil in gameplay. For player yaw ψ , pitch α , component offset $\mathbf{o}$ and recoil r ,

$$Q = R_y(-\psi - 90)R_x(\alpha), \quad \mathbf{p} = \mathbf{c} + Q(\mathbf{o} + (0, 0, .20r)), \quad M = T(\mathbf{p})R_y(-\psi - 90)R_x(\alpha)HS.$$

The held startup pose uses $\mathbf{c} = (0, 1.7, -5)$, $\psi = -90^\circ$, $\alpha = 3.8^\circ$. The body offset $(.32, -.25, -.68)$ becomes approximately $(.32, 1.49562, -5.69507)$. Every part follows the same orientation, so the whole gun aims coherently. Firing sets $r = 1$; it decays by $7\Delta t$. When $r > .65$, one extra emissive muzzle-flash cube is appended. Metal parts use specular $k_s = .75$ and exponent 80. Source: `Weapon.cpp`; these are cuboid barrels, not cylindrical meshes.

3.6 Shotgun and rifle assembly

Both long guns reuse the pistol's offset-to-world formula. The shotgun display anchor is $(-9, 1.65, -8)$; the rifle's is $(-5, 1.65, -8)$. Both have yaw -90° , pitch 0 and no recoil. Each assembly panel adds one cuboid; its row in the coordinate table gives the corresponding world transform.

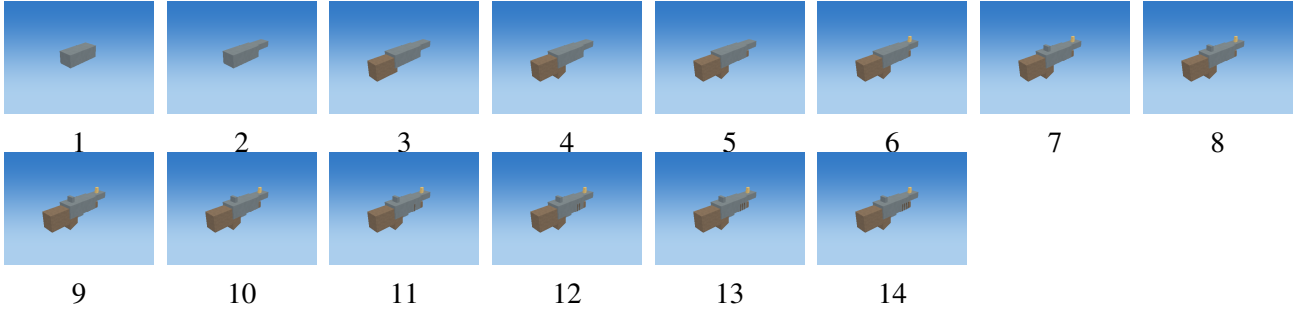


Figure 7: Shotgun: 14 cuboids, including the wider barrel, stock, fore-end, sights, trigger guard and four fore-end bands.

Step	Component	World center (m)	Scale (m)	Rotation (deg)
1	BODY	(-8.68,1.37,-8.79)	(0.29,0.25,0.7)	(0,0,0)
2	BARREL	(-8.68,1.43,-9.31)	(0.21,0.13,0.43)	(0,0,0)
3	STOCK	(-8.68,1.29,-8.28)	(0.24,0.3,0.45)	(0,0,0)
4	GRIP	(-8.68,1.15,-8.58)	(0.17,0.27,0.2)	(0,0,0)
5	FORE_END	(-8.68,1.3,-9.05)	(0.28,0.13,0.34)	(0,0,0)
6	FRONT_SIGHT	(-8.68,1.55,-9.34)	(0.05,0.13,0.07)	(0,0,0)
7	REAR_SIGHT	(-8.68,1.55,-8.59)	(0.13,0.1,0.06)	(0,0,0)
8	MUZZLE_INSET	(-8.68,1.43,-9.531)	(0.09,0.075,0.012)	(0,0,0)
9	TRIGGER_GUARD	(-8.68,1.17,-8.76)	(0.13,0.035,0.19)	(0,0,0)
10	GUARD_FRONT	(-8.68,1.25,-8.84)	(0.13,0.16,0.035)	(0,0,0)
11	DETAIL_0	(-8.68,1.31,-8.95)	(0.3,0.15,0.018)	(0,0,0)
12	DETAIL_1	(-8.68,1.31,-9.008)	(0.3,0.15,0.018)	(0,0,0)
13	DETAIL_2	(-8.68,1.31,-9.066)	(0.3,0.15,0.018)	(0,0,0)
14	DETAIL_3	(-8.68,1.31,-9.124)	(0.3,0.15,0.018)	(0,0,0)

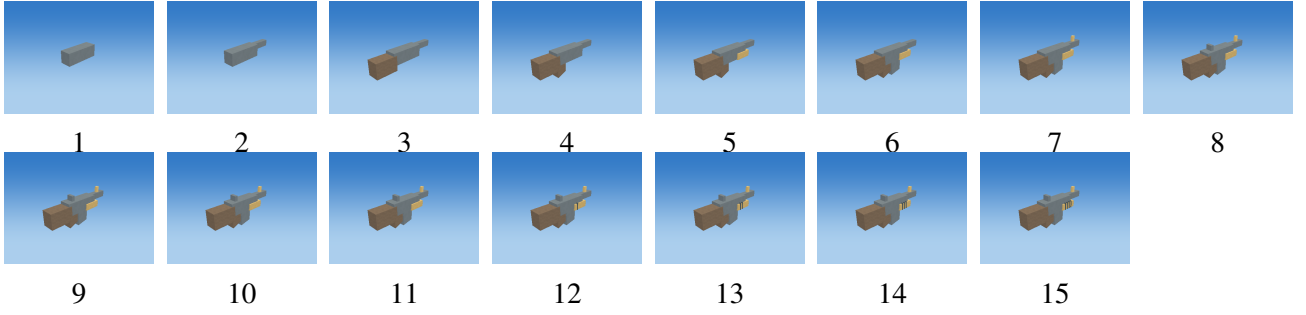


Figure 8: Rifle: 15 cuboids. The magazine is an additional sheared component; the narrower barrel and colored fore-end distinguish its silhouette.

Step	Component	World center (m)	Scale (m)	Rotation (deg)
1	BODY	(-4.68,1.37,-8.79)	(0.23,0.25,0.7)	(0,0,0)
2	BARREL	(-4.68,1.43,-9.31)	(0.11,0.13,0.43)	(0,0,0)
3	STOCK	(-4.68,1.29,-8.28)	(0.24,0.3,0.45)	(0,0,0)
4	GRIP	(-4.68,1.15,-8.58)	(0.17,0.27,0.2)	(0,0,0)
5	FORE_END	(-4.68,1.3,-9.05)	(0.28,0.13,0.34)	(0,0,0)
6	MAGAZINE	(-4.68,1.14,-8.83)	(0.15,0.35,0.24)	(0,0,0)
7	FRONT_SIGHT	(-4.68,1.55,-9.34)	(0.05,0.13,0.07)	(0,0,0)
8	REAR_SIGHT	(-4.68,1.55,-8.59)	(0.13,0.1,0.06)	(0,0,0)
9	MUZZLE_INSET	(-4.68,1.43,-9.531)	(0.09,0.075,0.012)	(0,0,0)
10	TRIGGER_GUARD	(-4.68,1.17,-8.76)	(0.13,0.035,0.19)	(0,0,0)
11	GUARD_FRONT	(-4.68,1.25,-8.84)	(0.13,0.16,0.035)	(0,0,0)
12	DETAIL_0	(-4.68,1.31,-8.95)	(0.3,0.15,0.018)	(0,0,0)
13	DETAIL_1	(-4.68,1.31,-9.008)	(0.3,0.15,0.018)	(0,0,0)
14	DETAIL_2	(-4.68,1.31,-9.066)	(0.3,0.15,0.018)	(0,0,0)
15	DETAIL_3	(-4.68,1.31,-9.124)	(0.3,0.15,0.018)	(0,0,0)

The long-gun grip has $h_{yz} = -.4$; the rifle magazine has $h_{yz} = -.22$. For the rifle magazine, scale $(.15, .35, .24)$ turns $(.5, .5, .5)$ into $(.075, .175, .12)$; shear makes $y = .175 - .22(.12) = .1486$. Adding its center $(-4.68, 1.14, -8.83)$ gives $(-4.605, 1.2886, -8.71)$. Shear changes the slant without introducing a different mesh. Shotgun fore-end uses Rubber, while the rifle's accent still follows the builder's material assignment. The magazine remains part of the same aimed assembly throughout recoil and camera rotation.

3.7 Circular targets and stands from 38 cubes

The representative Practice target is centered at $(0, 2.7, -19)$ with radius $r = .8$ m, thickness $.16$ m and yaw 0. A base, pole, two braces and two warning strips form the first six cubes. The remaining 32 horizontal slices approximate the circular plate.

$$h = 2r/32 = .05, \quad y_i = -r + (i + .5)h, \quad w_i = 2\sqrt{r^2 - y_i^2}, \quad (3)$$

$$\mathbf{p}_i = (0, 2.7 + y_i, -19), \quad S_i = (w_i, .05, .16), \quad R_i = R_y(\theta). \quad (4)$$

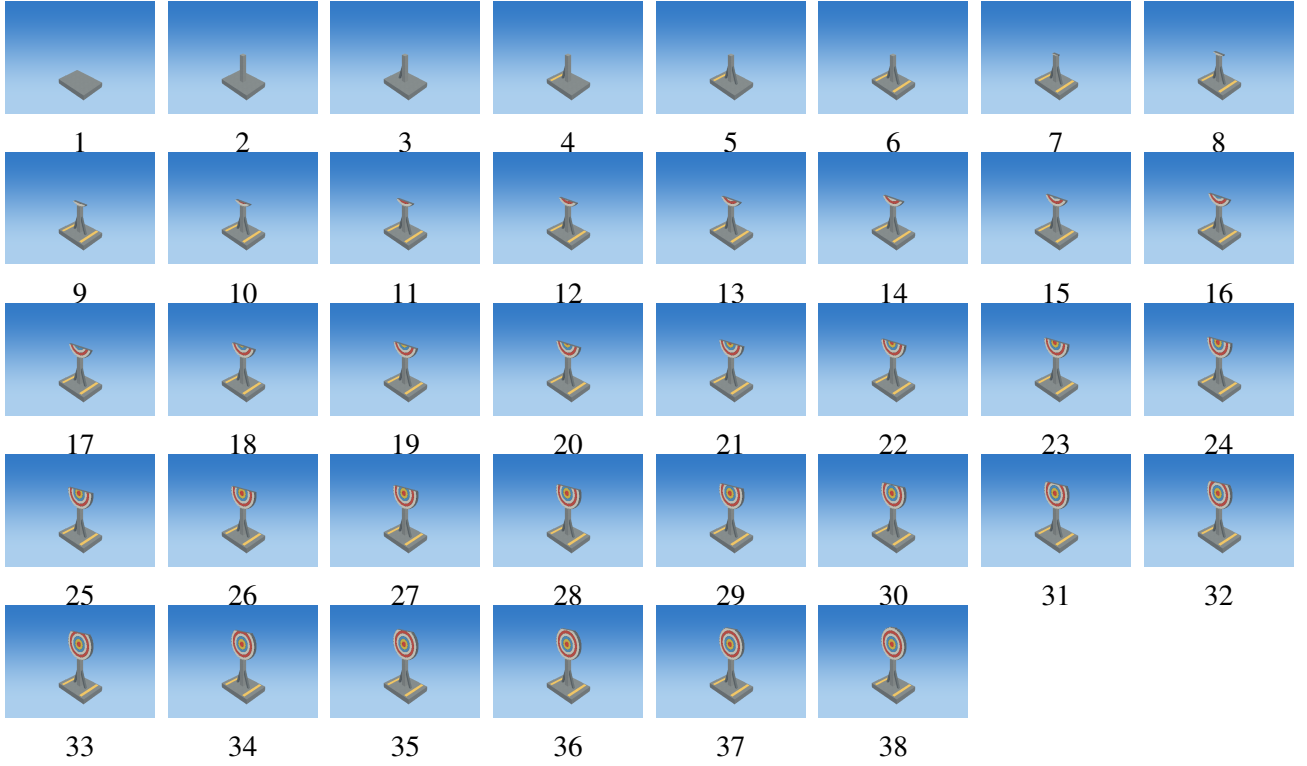


Figure 9: Target construction: 1 base, 2 pole, 3/5 braces, 4/6 strips, 7–38 plate slices. Rings are shader color.

Slice	Local y_i	Width w_i	World center
0	-.775	.396863	(0, 1.925, -19)
7	-.425	1.355544	(0, 2.275, -19)
15	-.025	1.599219	(0, 2.675, -19)
16	.025	1.599219	(0, 2.725, -19)
31	.775	.396863	(0, 3.475, -19)

The stand height is $H = \max(.3, 2.7 - .8) = 1.9$. Base: center (0, .15, -19), scale (2, .3, 1.5). Pole: (0, .95, -19), scale (.24, 1.9, .24). For side $q = \pm 1$, brace center is (.28 q , .751, -19), scale (.10, 1.064, .12), $R_z = 18q$ degrees; warning-strip center is (.7 q , .307, -19), scale (.16, .012, 1.36).

Ring continuity across slices. Each slice has scale vector $\mathbf{a} = (w_i/1.6, .05/1.6, 1)$ and offset vector $\mathbf{b} = (0, y_i/1.6, 0)$. These correspond to the uniforms `patternScale` and `patternOffset`. Then $\mathbf{q} = \mathbf{p}_l \odot \mathbf{a} + \mathbf{b}$ and $\rho = 2\|\mathbf{q}_{xy}\|$. The ring index is $\text{clamp}(\lfloor 6\rho \rfloor, 0, 5)$. Separators occur at $\text{fract}(6\rho) > .95$. Only local +Z faces print rings; others use (.15, .18, .21). Yaw rotates the slices, normals and scoring front together. Sources: `Target.cpp`, `object.frag`.

4 Animation, Shooting and Collision

4.1 Projectiles and exact hit detection

Projectile scales are (.10, .10, .25), (.08, .08, .18) and (.13, .13, .30) for pistol/shotgun/rifle. All use color (1, .8, .22), emission .6 and specular .8, and follow the muzzle-to-aim direction.

Weapon	Range (m)	Speed (m/s)	Cooldown (s)	Pellets	Spread
Pistol	25	42	.28	1	0
Shotgun	18	36	.80	9	.075
Rifle	70	65	.11	1	.009

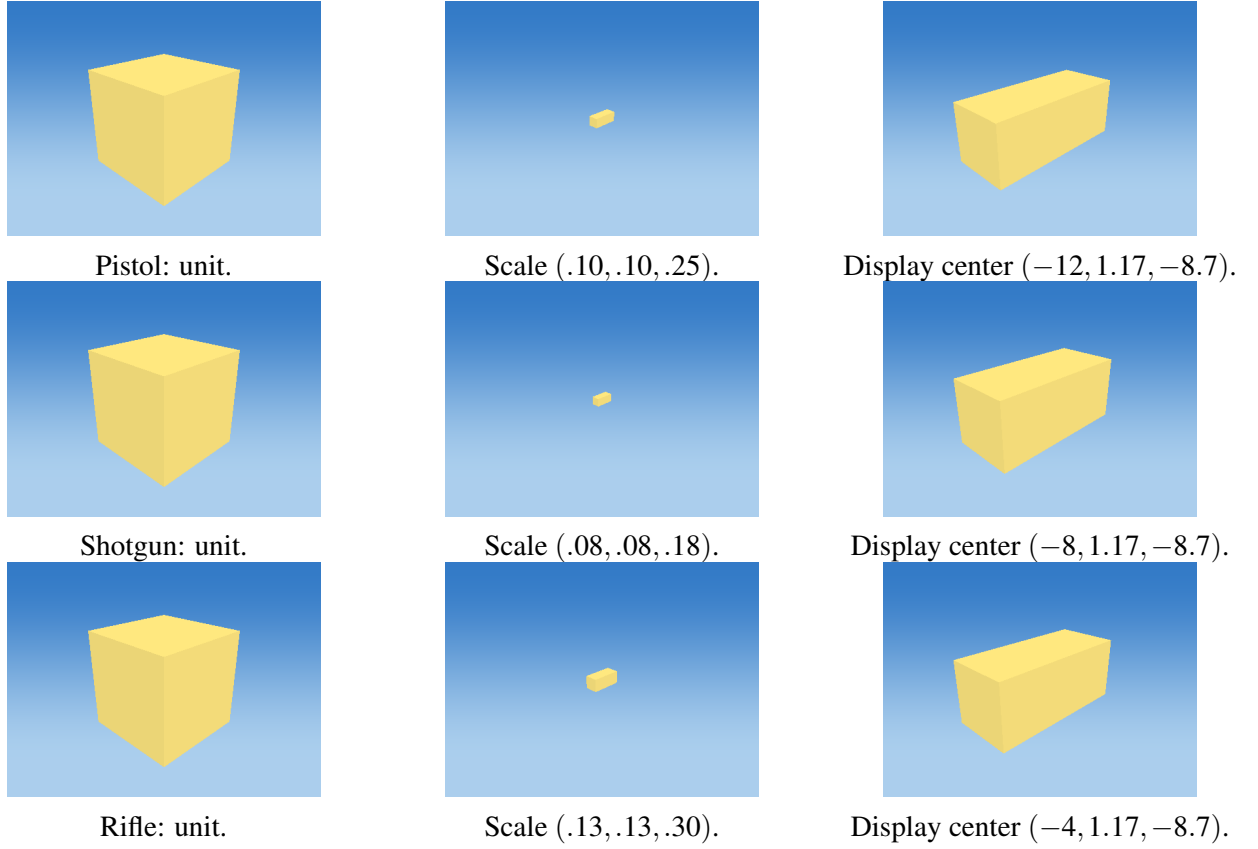


Figure 10: Three projectile models: unit cube, scaled cuboid and world placement. The display direction $(0, 0, -1)$ gives zero rotation.

The muzzle is $\mathbf{m} = \mathbf{c} + Q(.32, -.22, z_m)$, where $z_m = -1.02$ (pistol) or -1.52 (long guns). If the camera ray meets the nearest object at distance d_a , aim at $\mathbf{a} = \mathbf{c} + d_a \mathbf{f}$ and set $\mathbf{d} = \text{normalize}(\mathbf{a} - \mathbf{m})$. The shotgun emits one center pellet and eight with offsets $.075(\cos(2\pi i/8), \sin(2\pi i/8))$ in the ray's right/up basis. Rifle offsets are $.009(\sin(2.4n), \cos(1.7n))$ for shot count n .

Per update, travel is $\ell = \min(v\Delta t, \text{range} - \text{travelled})$, shortened to the nearest collision; $\mathbf{p} \leftarrow \mathbf{p} + \ell \mathbf{d}$. Simulation is subdivided to at most $1/120$ s. Visual angles are $\theta_x = \arcsin(d_y)$ and $\theta_y = \text{atan2}(-d_x, -d_z)$ in degrees.

Affine-box collision. Transform ray origin/direction by the inverse model basis: $\mathbf{o}_l = A^{-1}(\mathbf{o} - \mathbf{t})$, $\mathbf{d}_l = A^{-1}\mathbf{d}$ without renormalizing. Each axis intersects $[-.5, .5]$ using $t_1 = (-.5 - o_l)/d_l$, $t_2 = (.5 - o_l)/d_l$; the ray hits if the accumulated entry does not exceed exit. Parallel axes reject origins outside the slab. Targets test their actual slices after inverse yaw; NPCs test their animated cuboids. Front scoring additionally requires incoming local $d_z < 0$ and contact at $z = \text{thickness}/2$. This avoids treating the whole target's bounding square as its scoring circle.

4.2 People, birds and transient effects

NPCs share an offset transform: $\mathbf{p} = \mathbf{p}_{npc} + R_y(\text{heading})R_x(\text{fall})\mathbf{o}$, with component rotation $R_y(\text{heading})R_x(\text{fall})$. Humans are built from 21 cuboids; birds from 10. Their silhouettes and collision parts come from the same builders. Human height $H \in [2.05, 2.20]$ scales offsets/sizes by $q = H/1.82$. Reference torso offset is $(0, 1.12, 0)$ and size $(.46, .58, .25)$. Walk swing is $22\sin(\text{animation})$ degrees with opposite signs on the two sides. Falling uses $90\min(1, t/.75)^2$ degrees. Bird body size is $(.62, .26, .28)$; bob $b = .045\sin(\text{animation}/2)$ and flap $a_q = 35q\sin(\text{animation})$ degrees for side $q = \pm 1$. Wing center $(-.03, b - .35q\sin a_q, .35q\cos a_q)$ and size $(.38, .055, .64)$ produce visible flapping. Fatal bird hits use gravity 14 and pitch rate $260^\circ/\text{s}$. Sources: Human.cpp, Bird.cpp. **Effects equations.** Target flash uses $f = \text{hitTime}/.25$ and blends the ring color toward $(1, .76, .2)$ by $f/2$. Breaking emits

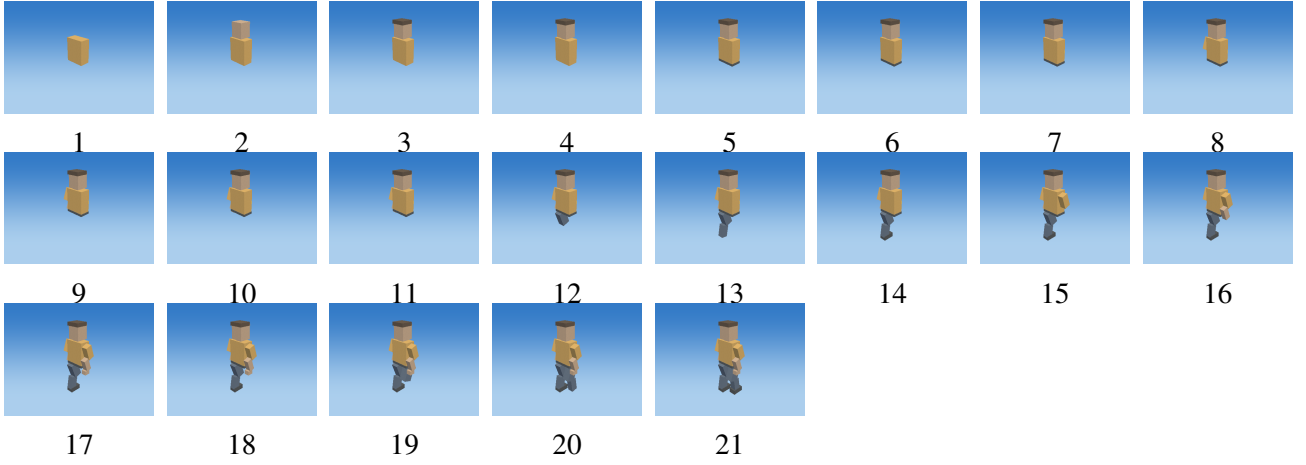


Figure 11: Human assembly: torso, head and details, followed by the two articulated arms and legs.

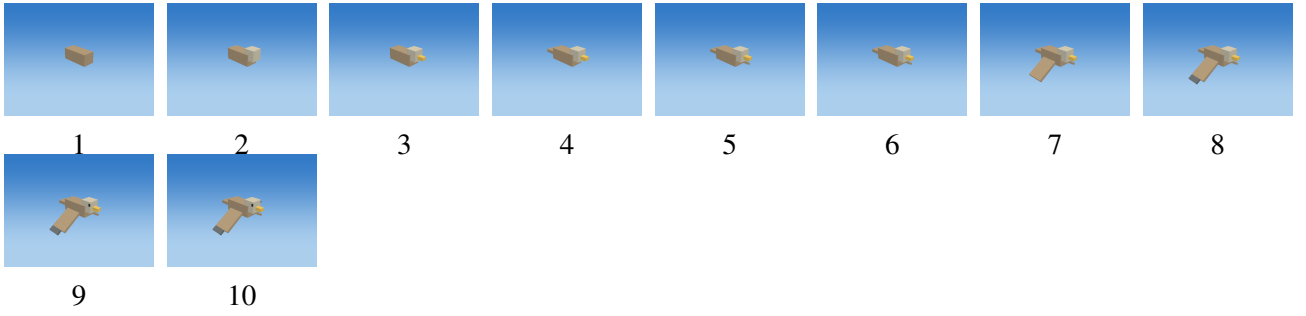
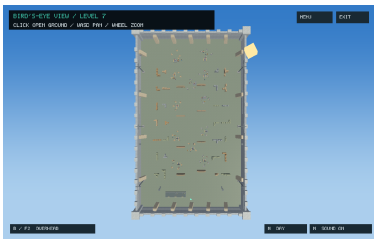


Figure 12: Bird assembly: body, head, beak, tail, wings, wing tips and eyes.

12 fragments with $\alpha_i = 2\pi i/12$, $\mathbf{v}_i = (2.4 \cos \alpha_i, 2 + 2 \sin \alpha_i, 1.5)$ and lifetime .75 s; gravity is 7. Blood uses gravity 13.5, seeded sizes .035–.07, and flattens near the floor. For celebration particle i , age $a = t - .045(i \bmod 7)$, angle $\beta = 2.39996i$ and speed $u = 5 + 1.4(i \bmod 5)$ give $\mathbf{p} = (0, 22, -26) + (ua \cos \beta, (5 + i \bmod 4)a - 2.5a^2, ua \sin \beta)$. Body grounding uses the lowest transformed corner. Cosmetic particles expire and do not award additional score.

5 Camera and User Interaction

5.1 Views and projection



Overhead: full arena and ground selection.



Observation: selected ground position at eye height.

Figure 13: Bird's-Eye navigation connects an overview ray to a valid ground-level observation point.

The same world can be examined from the player, elevated arena, side and free cameras. Bird's-Eye adds an independent overhead/observation pair and visible ground marker. Inspection cameras do not reposition the actual grounded player or permit firing from an arbitrary flying position. Given yaw ψ and pitch α , the camera forward direction is

$$\mathbf{f} = \text{normalize}(\cos \psi \cos \alpha, \sin \alpha, \sin \psi \cos \alpha).$$

Mouse deltas give $\psi \leftarrow \psi + .12\Delta x$ and $\alpha \leftarrow \text{clamp}(\alpha - .12\Delta y, -89, 89)$. Let $\mathbf{s} = \text{normalize}(\mathbf{f} \times \mathbf{u}_0)$, $\mathbf{u} = \mathbf{s} \times \mathbf{f}$, $\mathbf{u}_0 = (0, 1, 0)$ and eye $\mathbf{e}$. Then

$$V = \begin{bmatrix} s_x & s_y & s_z & -\mathbf{s} \cdot \mathbf{e} \\ u_x & u_y & u_z & -\mathbf{u} \cdot \mathbf{e} \\ -f_x & -f_y & -f_z & \mathbf{f} \cdot \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad P = \begin{bmatrix} k/a & 0 & 0 & 0 \\ 0 & k & 0 & 0 \\ 0 & 0 & (F+n)/(n-F) & 2Fn/(n-F) \\ 0 & 0 & -1 & 0 \end{bmatrix},$$

where $k = 1/\tan(FOV/2)$, aspect $a = w/h$, and n, F are near/far planes. Gameplay uses a 60° perspective; isolated demonstration images use 45° and automatic framing so small parts remain visible. These are perspective views, not orthographic projections.

5.2 Interactive controls and movement

Key/button	Action	Function/logic	Visible effect
WASD, Shift	Walk/sprint	Ground movement and collision	Viewpoint moves at 5/9 m/s
Mouse	Aim	Camera::look	Yaw/pitch and held gun turn
Click/Space; 1/2/3	Fire; weapon	Game::fire; session action	Projectile/recoil; gun changes
F1-F4; Q/E	Views; fly	setCamera; free movement	Player/arena/side/free view
F6/F7/F8	Shading	shadingMode=0/1/2	Flat/Gouraud/Phong lighting
N; M	Night; audio	Session actions	Light rig / sound changes
Esc/Enter/R; Tab	Session; pointer	Session action; input loop	Pause/confirm/restart; cursor
Click/wheel/B	Bird's-Eye	Ground picking / zoom	Observation/overhead view
F5	Snapshot	CSV request in runScene	Calculation file updates

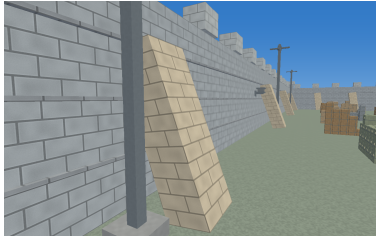
The input loop is `runScene` in `Application.cpp`. Ground movement flattens and normalizes the forward vector. Collision resolves X/Z separately in steps at most .15 m; the player remains within $x \in [-26, 26]$, $z \in [-96, -4]$. Free-camera speeds are 12/30 m/s and height is at least .3 m.

Picking. The overhead camera starts at $(0, 120, -50)$, yaw -90° , pitch -89.9° . For normalized cursor (u, v) , ray $\mathbf{r} = \text{normalize}[\mathbf{f} + (2u - 1)a \tan 30^\circ \mathbf{s} + (1 - 2v) \tan 30^\circ \mathbf{u}]$. Its ground intersection is $\mathbf{e} - \mathbf{r}(e_y/r_y)$. The game rejects unsafe/occluded positions, then places observation eye 1.7 m above valid ground. The marker uses scale $(.9, .08, .9)$, center $(x, .045, z)$; its heading cube follows observation yaw/pitch.

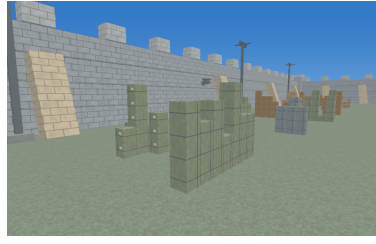
6 Texture Techniques

6.1 Generation, mapping and sampling

Nine original grayscale reflectance maps are generated offline by `tools/generate_textures.cpp`; they are not photographs or per-frame procedural shaders. The shipped P6 PPM files contain 256×256 RGB pixels. Runtime loads them once into a `GL_TEXTURE_2D_ARRAY` with `GL_RGB8` storage. **Offline pattern formulas.** Integer hash noise starts with $n = (x \& 255)374761393 + (y \& 255)668265263 + \text{seed}2246822519$ modulo 2^{32} , scrambles $n \leftarrow (n \oplus (n \gg 13))1274126177$, then returns $((n \oplus (n \gg 16)) \& 65535)/65535$. A wrapped grid interpolates noise using $g(t) = t^2(3 - 2t)$ in both dimensions. Masonry staggers 128-pixel bricks on 64-pixel courses; wood uses $\sin(.49x + 3 \sin(2\pi y/256) + 2c)$ and plank seams every 64 pixels. Concrete/gravel combine coarse/fine noise; metal varies mainly along rows; fabric alternates $(x + y) \bmod 2$; rubber adds an 8-pixel grip grid. Values are clamped to $[0, 1]$, multiplied by 255 and copied to RGB.



Masonry: repeating bricks and mortar.



Cargo: wood grain and bands.



Equipment: material and scale contrast.

Figure 14: Surface textures on the assembled arena and equipment. Geometry controls silhouette; face-local UV coordinates control surface repetition.

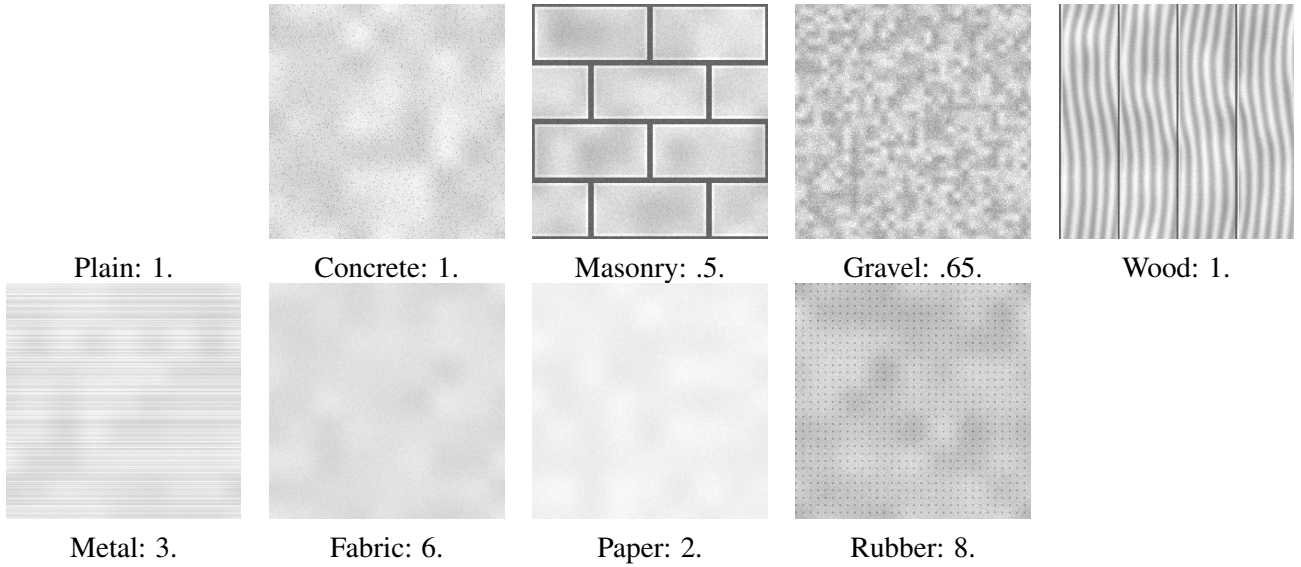


Figure 15: The actual texture pixels; tiling values are repeats per scaled local meter, not image enlargement factors.

Face-local UVs. For local position $\mathbf{p}_l$, scale $\mathbf{s}$ and tiling τ , let $\mathbf{q} = (\mathbf{p}_l + .5) \odot |\mathbf{s}| \tau$. Use (q_x, q_z) on Y faces, (q_z, q_y) on X faces and (q_x, q_y) on Z faces. A 60 m floor with $\tau = .65$ repeats 39 times across X and 65 times across Z. Because UVs derive from local position, rotation/translation do not cause texture swimming; shear stretches the pattern with the surface. This is planar face mapping, not blended triplanar mapping.

Sampling and color. Wrapping is GL_REPEAT; magnification is linear and minification is trilinear mipmap filtering. Mip levels are generated once. Base color is $\mathbf{B} = \mathbf{C}_{surface} \odot \text{texture}(UV, layer)$ before lighting. Emission $> .5$ selects Plain; target-pattern surfaces select Paper; otherwise the assigned material is used. Maps are linear multipliers, with no automatic sRGB decoding. Nine base images occupy about 1.69 MiB; with mipmaps nominal RGB storage is about 2.25 MiB. There are no normal, roughness or displacement maps.

7 Lighting and Shading

7.1 Ambient, directional and material lighting

The shader computes Phong reflection [3] from the actual world position and inverse-transpose normal. Let $\mathbf{N}$ be the normalized normal, $\mathbf{V} = \text{normalize}(\text{eye} - \text{position})$ and $\mathbf{L}$ point toward the light. A light's effective RGB intensity $\mathbf{C}$

already includes any attenuation/cone factor.

$$a = \max(\mathbf{N} \cdot \mathbf{L}, 0), \quad \mathbf{R} = 2(\mathbf{N} \cdot \mathbf{L})\mathbf{N} - \mathbf{L}, \quad (5)$$

$$\mathbf{D} = \mathbf{A} + \sum_j \mathbf{C}_j a_j, \quad \mathbf{S} = \sum_{j: a_j > 0} \mathbf{C}_j k_s \max(\mathbf{R}_j \cdot \mathbf{V}, 0)^n. \quad (6)$$

In textbook notation, $I_a = k_a I_L$, $I_d = k_d I_L \max(\mathbf{N} \cdot \mathbf{L}, 0)$ and $I_s = k_s I_L \max(\mathbf{R} \cdot \mathbf{V}, 0)^n$. Here base RGB supplies ambient/diffuse reflectance; materialSpecular and shininess supply k_s, n . Day ambient is (.30, .32, .35); night is (.012, .016, .025). Sun direction is normalize(.45, .8, .3) $\approx$ (.466002, .828449, .310668), with day color (.95, .88, .73) and zero night intensity. It has no distance attenuation. For the upward floor normal (0, 1, 0), the directional Lambert



(a) Day ambient only, mask 1.



(b) Directional sun only, mask 2.



(c) Complete day preset, mask 15.



(d) Complete night preset, mask 15.

Figure 16: Ambient fills all surfaces; the directional sun establishes orientation. Day and night use different source intensities.

factor is .828449. Before texture/specular, day diffuse is approximately

$$(.30, .32, .35) + (.95, .88, .73)(.828449) = (1.087026, 1.049035, .954768).$$

Values may exceed one before the final clamp; there is no HDR tone-mapping operator. Default material values are $k_s = .12, n = 24$; weapons .75, 80; target plates/fixtures .45, 48; crate-body specular .09 and human .06.

$$\mathbf{L}_{lit} = \mathbf{B} \odot \mathbf{D} + \mathbf{S}, \quad \mathbf{C}_{out} = [\text{clamp}((1 - e)\mathbf{L}_{lit} + e\mathbf{B}, 0, 1)]^{1/2.2}.$$

Emission e blends toward unlit base color; it does not cast light onto nearby objects. The daytime sun is an emissive cube at (25, 35, -80), scale (4, 4, 4) and rotation (0, 25, 15)°. It disappears at night; no moon is drawn. Lens emission is 1 at night and 0 by day. The sky is a separate fullscreen gradient $\text{mix}(\text{horizon}, \text{zenith}, \text{smoothstep}(.15, 1, uv_y))$, independent of scene lighting and camera orientation. **Shader connection.** The following statements are from `lighting.glsl`; attenuation is already included in color when this function is called.

```
float ndotl=max(dot(N,L),0.0);
diffuse+=color*ndotl;
vec3 R=reflect(-L,N);
```

When `ndotl>0.0`, the next statement adds `color*materialSpecular*pow(max(dot(R,V),0.0),shininess)` to the specular accumulator. The explicit guard prevents highlights on a

back-facing surface.

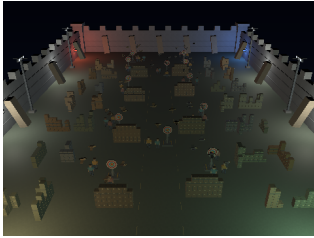
7.2 Point lamps, stadium spots and attenuation

Night uses eight point lamps and six stadium spotlights. A visible fixture is geometry; the light position/direction is a separate uniform. Each stadium fixture has 12 cuboids and three lenses but only *one* spotlight. Day point/spot colors are zero.

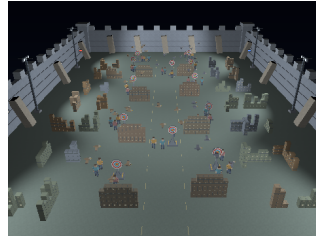
$$\delta = \mathbf{p}_{light} - \mathbf{p}, \quad d = \max(\|\delta\|, .001), \quad \mathbf{L} = \delta/d,$$

$$A_p = \frac{1}{1 + .09d + .032d^2}, \quad A_s = \frac{1}{1 + .025d + .002d^2}.$$

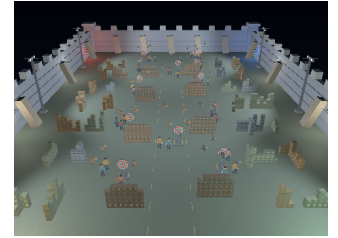
For a spot, $c = \text{dot}(-\mathbf{L}, \mathbf{d}_{spot})$, $t = \text{clamp}((c - \cos 53^\circ)/(\cos 32^\circ - \cos 53^\circ), 0, 1)$ and cone intensity $I = t^2(3 - 2t)$. Thus the inner cone is fully lit, the penumbra is smooth and the outer region receives zero from that spotlight. Other lights may still illuminate it. **Point example.** Directly under the warm lamp $(-18, 4.7, -12)$, floor point



(a) Point family only.



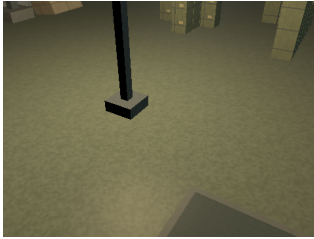
(b) Spot family only.



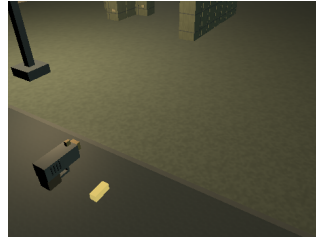
(c) Ambient+points+spots.

Figure 17: Night source isolation at a fixed camera; combining the families fills different regions of the same arena.

$(-18, 0, -12)$ has $d = 4.7$, $\mathbf{L} = (0, 1, 0)$ and $A_p = 1/(1 + .423 + .70688) = .469510$ approximately. Its diffuse increment is $(2.2, 1.65, .9)A_p \approx (1.032922, .774692, .422559)$. This contribution joins the other light terms before texture modulation, specular reflection, emission and gamma. For horizontal offset x from the lamp's floor projection,



Floor offset: 1 m.



Floor offset: 5 m.



Floor offset: 12 m.

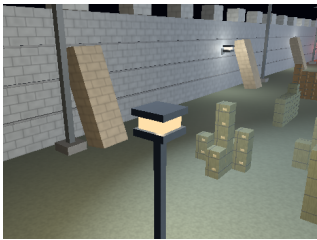
Figure 18: Point-light falloff across the floor; both attenuation and the normal–light angle reduce the diffuse term with horizontal distance.

$$d(x) = \sqrt{x^2 + 4.7^2}, \quad \mathbf{N} \cdot \mathbf{L} = \frac{4.7}{d(x)}, \quad \Delta \mathbf{D} = \frac{(2.2, 1.65, .9) 4.7}{d(x)[1 + .09d(x) + .032d(x)^2]}. \quad (7)$$

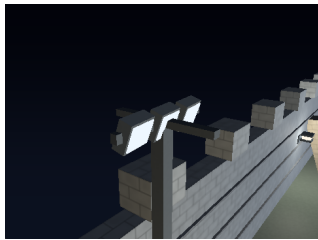
The spotlight's transition has zero derivative at both cone boundaries because $I'(t) = 6t(1 - t)$. At $c = (\cos 32^\circ + \cos 53^\circ)/2$, $t = .5$ and $I = .5$; this is the midpoint in cosine space, not necessarily the midpoint angle. Distance attenuation then multiplies this cone intensity.

Point positions are $(\pm 18, 4.7, -12)$, $(\pm 28.8, 5, -38)$, $(\pm 28.8, 5, -68)$; the rear pair is $(\pm 28.8, 3.2, -89)$. Colors in source order are $(2.2, 1.65, .9)$, $(1.2, 2, 1.3)$, four $(1.7, 1.6, 1.4)$, then red $(2, .25, .15)$ and blue $(.25, .65, 2)$ accents. Spots sit at $(\pm 28, 12, z)$ for $z = -16, -48, -80$, point along $\text{normalize}(-x, -10, -4)$ and have color $(1.8, 1.9, 2.1)$.

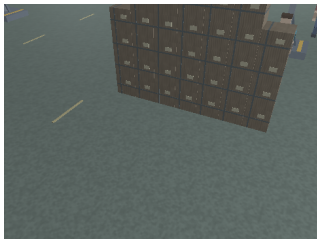
Fixture construction uses pole center $(x, 6, z)$ and size $(.38, 12, .38)$; heads use rotation $(-\arcsin(d_y), \text{atan2}(d_x, d_z), 0)$ in degrees. Each lens is displaced $.18d$ from its head center. The general matrix and offset rules already demonstrated for weapons apply unchanged.



Point fixture: warm local lamp.



Stadium fixture: three lenses.

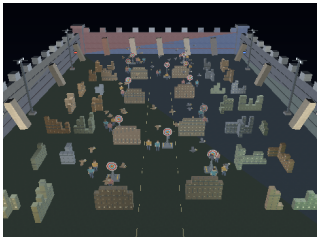


Illumination along the spotlight axis.

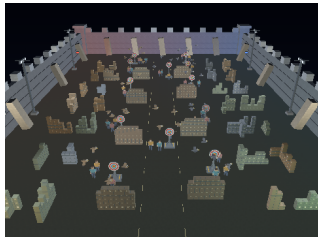
Figure 19: Visible light fixtures and their illumination. Lens emission brightens the fixture itself; point/spot uniforms illuminate other surfaces.

7.3 Flat, Gouraud and Phong shading

All modes use the same Phong *reflection equation*; the difference is where lighting is evaluated and how the result is interpolated [3, 4]. With separate cube face normals, Phong shading does not round the cube’s geometric edges. Texture lookup and target-ring evaluation remain per fragment in every mode.



(d) Night: flat.



(e) Night: Gouraud.



(f) Night: Phong.



(g) Flat close view.



(h) Gouraud close view.



(i) Phong close view.

Figure 20: Controlled shading comparison: each row holds scene, camera and source mask fixed. Large floor/wall triangles expose the difference under local night lights.

Mode	Evaluation and interpolation	Consequence
Flat (F6)	Lighting evaluated at vertices; flat passes the provoking vertex result unchanged over the triangle.	A local source may produce a constant triangle-sized region.
Gouraud (F7)	Diffuse and specular computed at vertices and interpolated across the primitive.	Smooth vertex-to-vertex variation, but a highlight between vertices can be missed.
Phong (F8)	Interpolate position/normal; normalize and evaluate lighting in the fragment shader.	Distance, cone and highlights respond at fragment resolution; default mode.

For a smooth varying a , the rasterizer's perspective-correct interpolation has the form $a = (\sum_i \lambda_i a_i / w_i) / (\sum_i \lambda_i / w_i)$; flat varyings bypass this interpolation [2]. The implementation assigns both smooth and flat diffuse/specular outputs in `object.vert`; `object.frag` selects them for modes 0/1 or calls `lighting(worldPosition, worldNormal, ...)` for mode 2.

Coverage. Two presets, three shading modes, sixteen source masks and two viewpoints form $2 \times 3 \times 16 \times 2 = 192$ lighting comparisons. Bits are `ambient=1`, `sun=2`, `points=4`, `spots=8`. Inactive families produce identical images: `points/spots` by day, `sun` by night. Sky and emissive geometry remain visible without these sources. Shadows, AO, bloom and PBR are not implemented.

8 Implementation, Advanced Features and Verification

GLFW provides context/input; GLAD loads OpenGL functions [5]. In `src/core/`, `runScene` handles input and frame timing. `Game::update/fire` manage gameplay; `world/level` builders generate geometry and trajectories. `Renderer::drawArena` submits the scene, and `drawTransformedCube` uploads each model/normal matrix. `TextureCache` owns textures; `shaders/` implements projection and illumination. `Persistence/UI` code owns records, menus and HUD. Each frame processes input, updates simulation, constructs the scene, renders world/sky and overlays UI.

8.1 Build and execution

Use 64-bit MinGW/C++17 and an OpenGL 3.3 driver. Run `build.bat`, then `main.exe`; the portable release is `build/release/TargetShooter.exe`. Keep `assets/` and `shaders/` alongside it.

8.2 Advanced features and practical optimizations

A shared cube VBO avoids repeated geometry uploads. Cached uniforms/rigs, skipped zero-color lights and reusable scene vectors reduce work. Texture layers share one array binding; CPU normal matrices avoid per-vertex inverses. Each cube still requires its own draw call. Additional features include deterministic cargo generation, reachability validation, animated NPC penalties, Bird's-Eye picking, persistent leaderboards and asynchronous calculation CSV snapshots.

8.3 Parameters for a live demonstration

Changing target amplitude in `Level*.cpp` widens its path; frequency changes speed, while spin changes front-face orientation. In `Weapon.cpp`, part scales change the silhouette and grip shear changes its slope. In `Lighting.cpp`, light color scales brightness and direction moves the spot axis. In `lighting.glsl`, a larger shininess exponent narrows highlights; larger attenuation coefficients shorten a lamp's reach. Increasing texture tiling repeats the pattern more frequently. These parameters control the visible geometry, motion and surface response independently.

8.4 Verification and reproducibility

Existing CPU tests cover transforms, shears, collision, ring damage, seeded layouts and session persistence. OpenGL smoke checks cover views, weapons and day/night/shading. Repeated demonstration builds produced identical images on the same GPU; inactive lighting masks also matched. These checks establish functional behavior, not universal performance. Different GPU/driver rasterizers may produce small pixel differences.

9 Conclusion and Demonstration

The game connects cube assembly, transformations, cameras, textures and lighting to an interactive seven-level challenge. Its strongest result is traceability: the same builder values define rendered geometry, demonstrations and collision. Circular targets remain slice approximations; barrels and bullets remain cuboids. There is no skinning, Boolean mesh union, shadow map, PBR or ray-traced rendering. Ray-box intersection is used for collision and picking only. Future work could add curved meshes, instancing and shadows without replacing the shared source of geometry.

9.1 Demonstration sequence and key concepts

Level 1 demonstrates static aiming, Level 2 introduces translation, and Level 7 combines moving targets, cover and NPC penalties. Weapon switching compares range, spread and recoil; camera switching exposes the same scene from different positions. The night floor reveals the flat, Gouraud and Phong differences through F6/F7/F8. **Model composition:** with column vectors, $TR_zR_yR_xHS$ scales locally before world placement. **Normal transformation:** the inverse transpose preserves normal-tangent perpendicularity under shear and nonuniform scale. **Shading:** Gouraud can miss a highlight when no vertex samples its peak. **Collision:** segment intersection detects obstacles between successive projectile positions. **Light isolation:** masks that differ only in a zero-intensity source family produce the same illumination.

9.2 Source and image attribution

Game graphics, models and texture patterns are by Kazi Rifat Al Muin [1]. The university emblem identifies Khulna University of Engineering & Technology [6].

References

- [1] K. R. Al Muin, “3D Target Shooter,” GitHub, 2026. Accessed: Oct. 8, 2026. [Online]. Available: <https://github.com/KaziRifatAlMuin/Graphics-Target-Shooter-Game>
- [2] M. Segal and K. Akeley, *The OpenGL Graphics System: A Specification, Version 3.3 (Core Profile)*. Khronos Group, Mar. 11, 2010. [Online]. Available: <https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf>
- [3] B. T. Phong, “Illumination for computer generated pictures,” *Commun. ACM*, vol. 18, no. 6, pp. 311–317, Jun. 1975, doi: 10.1145/360825.360839.
- [4] H. Gouraud, “Continuous shading of curved surfaces,” *IEEE Trans. Comput.*, vol. C-20, no. 6, pp. 623–629, Jun. 1971, doi: 10.1109/T-C.1971.223313.
- [5] GLFW, “Getting started,” *GLFW 3.3 Documentation*. Accessed: Oct. 8, 2026. [Online]. Available: <https://www.glfw.org/docs/3.3/quick.html>
- [6] Khulna University of Engineering & Technology, “KUET.” Accessed: Oct. 8, 2026. [Online]. Available: <https://www.kuet.ac.bd/>